Architecture

CryptoPulse – System Architecture

Version: 1.0

---

1. Overview

CryptoPulse is an automated data engineering platform that collects cryptocurrency market data from multiple public APIs, validates and transforms the data through a layered ETL architecture, stores historical datasets in PostgreSQL, and exposes analytics-ready datasets to Power BI dashboards.

The architecture follows a modular, layered design that separates extraction, validation, transformation, storage, orchestration, analytics, and visualization. Each component has a single responsibility, making the system maintainable, testable, and scalable.

---

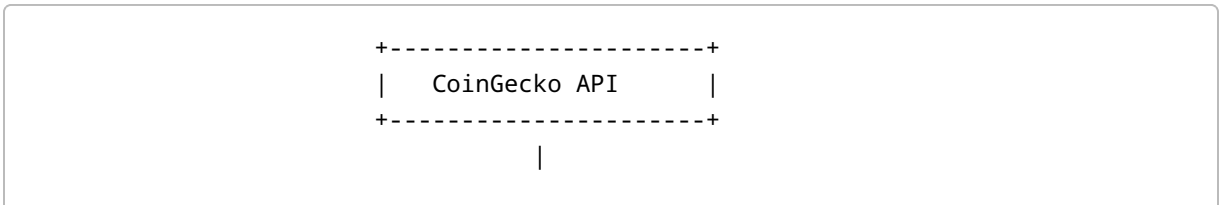
2. Architecture Principles

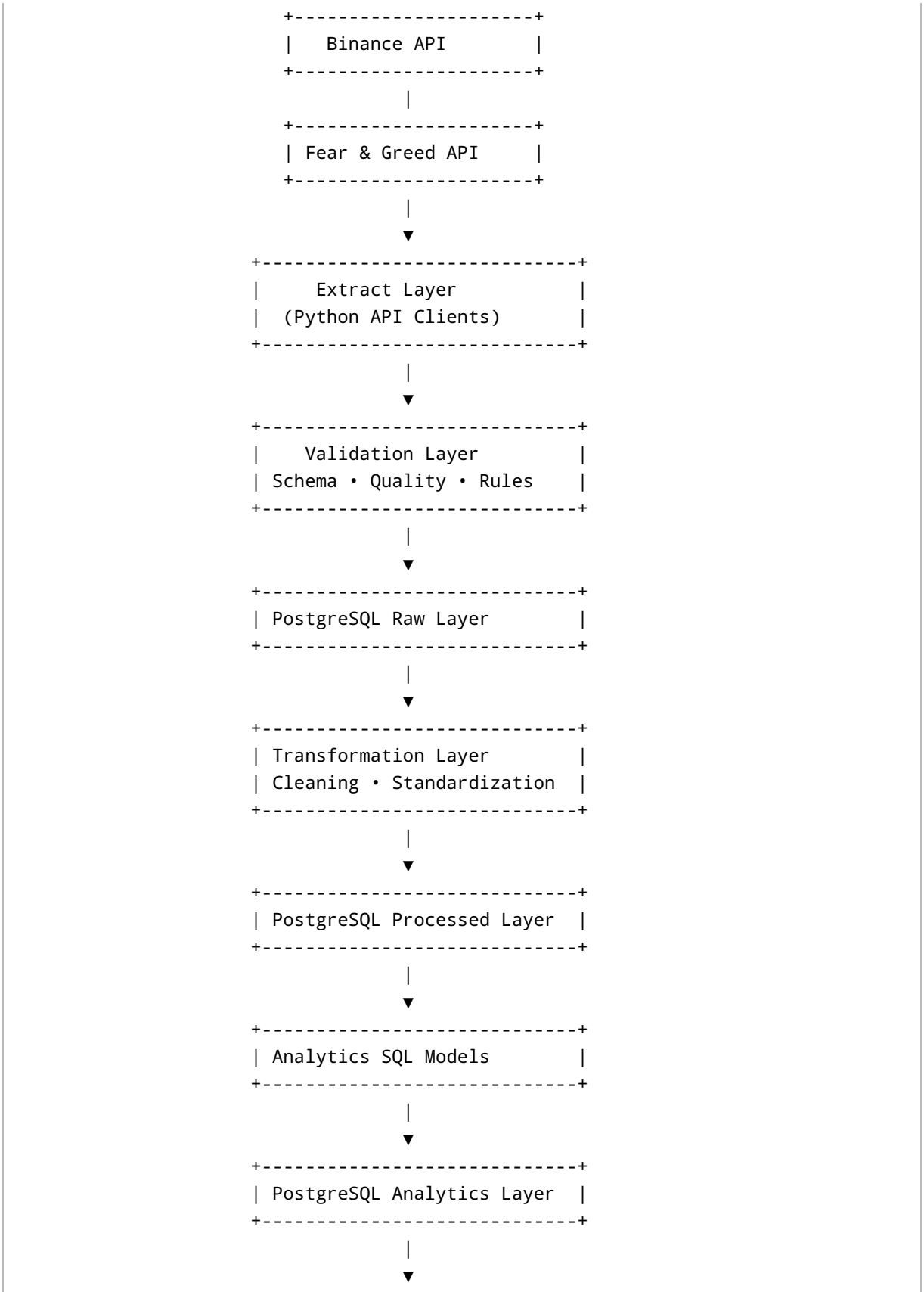
The system is designed around the following principles:

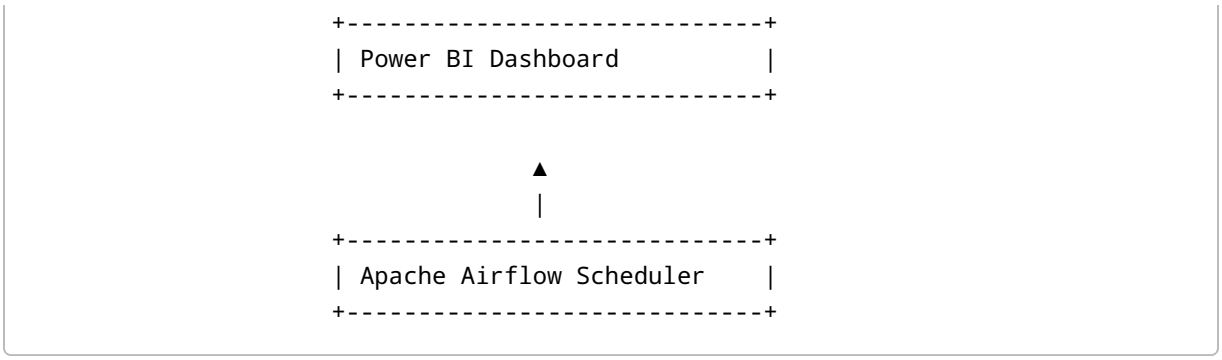
- Modular ETL components
- Separation of concerns
- Immutable raw data storage
- Incremental data processing
- Idempotent pipeline execution
- Configuration-driven behavior
- Fault tolerance with retries
- Comprehensive logging and monitoring
- Analytics-optimized data models
- Containerized deployment

---

3. High-Level Architecture




Binance API
Fear & Greed API
Extract Layer
(Python API Clients)
Validation Layer
Schema • Quality • Rules
PostgreSQL Raw Layer
Transformation Layer
Cleaning • Standardization
PostgreSQL Processed Layer
Analytics SQL Models
PostgreSQL Analytics Layer



---

4. Component Architecture

4.1 Data Sources

External APIs provide market and sentiment data.

CoinGecko

- Coin metadata
- Prices
- Market capitalization
- Trading volume
- Historical prices
- Trending coins

Binance

- OHLC candles
- Latest price
- Exchange trading volume

Fear & Greed Index

- Current sentiment
- Historical sentiment

4.2 Extraction Layer

Responsibilities

- Call external APIs
- Handle authentication (if required)
- Respect API rate limits

- Retry failed requests
- Measure response time
- Normalize responses into DataFrames
- Log API activity

Output

Raw datasets ready for validation.

4.3 Validation Layer

Incoming datasets pass through validation before persistence.

Validation includes:

Schema Validation

- Required columns
- Expected data types
- Missing fields

Data Validation

- Null values
- Invalid timestamps
- Currency validation
- Numeric precision

Business Rules

Reject records containing:

- Negative prices
- Negative market capitalization
- Negative trading volume

Duplicate Detection

Composite key:

```
coin_id
timestamp
source
```

Anomaly Detection

Flag:

- Large price spikes
- Abnormal volume changes
- Missing scheduled snapshots

4.4 Raw Layer

Purpose:

Store immutable API responses exactly as received.

Characteristics:

- Append-only
- No transformations
- Historical snapshots
- Source metadata
- Audit trail

Primary tables:

- raw_coin_market
- raw_exchange_prices
- raw_fear_greed
- api_request_logs
- etl_run_logs

4.5 Transformation Layer

Responsibilities:

- Standardize schemas
- Normalize timestamps (UTC)
- Convert data types
- Remove duplicates
- Handle missing values
- Calculate derived metrics

Output:

Clean business-ready datasets.

4.6 Processed Layer

Purpose:

Store validated and standardized market data.

Characteristics:

- Clean schema
- Consistent naming
- Incremental updates
- Analytics-ready

4.7 Analytics Layer

Purpose:

Provide optimized reporting tables.

Examples:

- Market overview
- Coin summary
- Top gainers
- Top losers
- Historical prices
- Volume trends
- Sentiment analysis
- Pipeline health

These tables are optimized for BI queries and dashboard performance.

---

5. ETL Pipeline Architecture

```
Extract
    │
    ▼
Validate
    │
    ▼
Load Raw
```

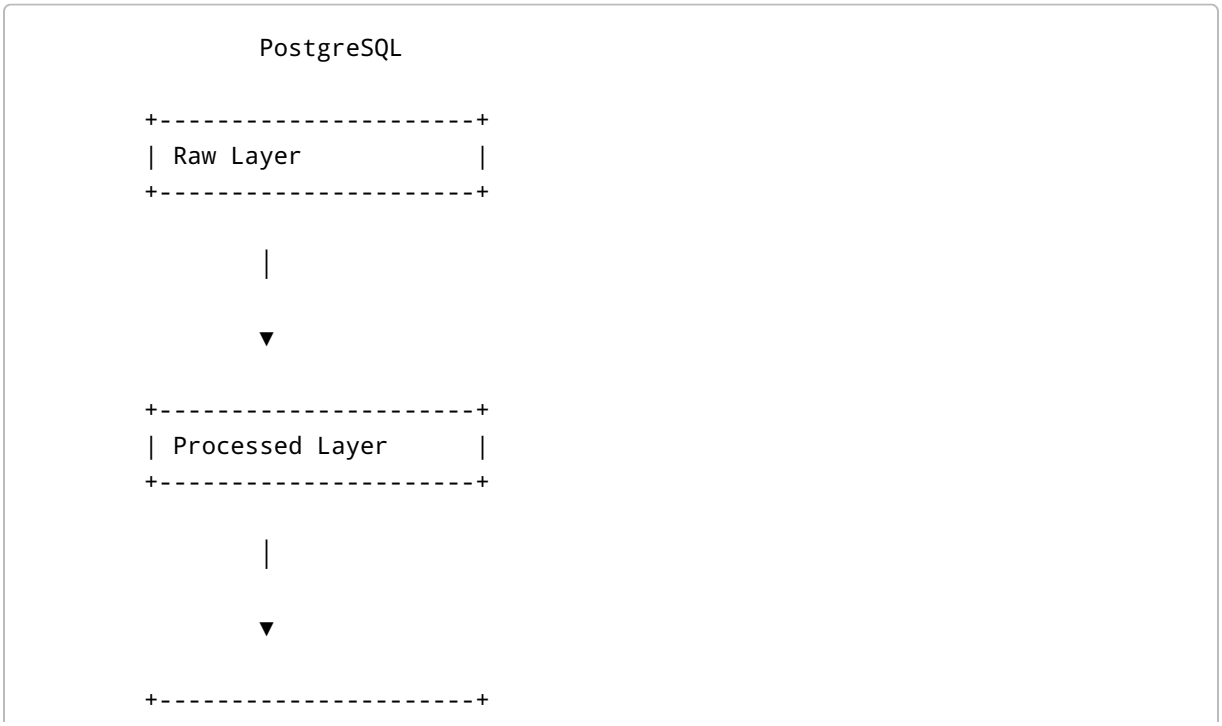
```
    |
    ▼
Transform
    |
    ▼
Load Processed
    |
    ▼
Build Analytics
    |
    ▼
Run Quality Checks
    |
    ▼
Generate Logs
    |
    ▼
Refresh Dashboard
```

Each stage is isolated and can be tested independently.

---

6. Database Architecture

The PostgreSQL database is organized into three logical layers.



```
        | Analytics Layer      |
        +----------------------+
```

Raw Layer

Stores immutable API snapshots.

Processed Layer

Stores validated and transformed data.

Analytics Layer

Stores aggregated datasets for reporting.

---

7. Airflow Architecture

Apache Airflow orchestrates the entire workflow.

DAG Tasks

```
extract_market_data

↓

validate_data

↓

load_raw_data

↓

transform_data

↓

load_processed_data

↓

build_analytics
```

```
↓

run_quality_checks

↓

generate_logs

↓

refresh_dashboard
```

Responsibilities

- Scheduling
- Dependency management
- Retry logic
- Failure recovery
- Execution history
- Monitoring

---

8. Incremental Loading Strategy

The pipeline only loads new records.

Composite uniqueness:

```
coin_id
timestamp
source
```

Execution flow:

```
Read latest timestamp

↓

Extract latest data

↓

Compare existing records
```

```
↓

Insert unseen rows

↓

Commit transaction
```

Benefits:

- Faster execution
- Reduced storage
- No duplicate history
- Efficient scheduling

---

9. Logging Architecture

Logging exists at two levels.

API Logging

Captures:

- Endpoint
- Duration
- Status code
- Retry count
- Timestamp

Pipeline Logging

Captures:

- Pipeline ID
- Task name
- Runtime
- Rows extracted
- Rows loaded
- Validation failures
- Error messages
- Execution status

Logs support troubleshooting, auditing, and performance monitoring.

10. Data Quality Architecture

Validation occurs before data is written to the processed layer.

Checks include:

- Required columns
- Data types
- Duplicate rows
- Null values
- Business rule validation
- Timestamp validation
- Numeric precision
- Missing snapshots
- Outlier detection

Invalid records are rejected or flagged according to severity.

---

11. Power BI Architecture

Power BI connects only to the Analytics Layer.

```
Analytics Tables

↓

Views

↓

Power BI Dataset

↓

Dashboard Pages
```

Report sections include:

- Executive Overview
- Market Overview
- Price Analysis
- Volume Analysis

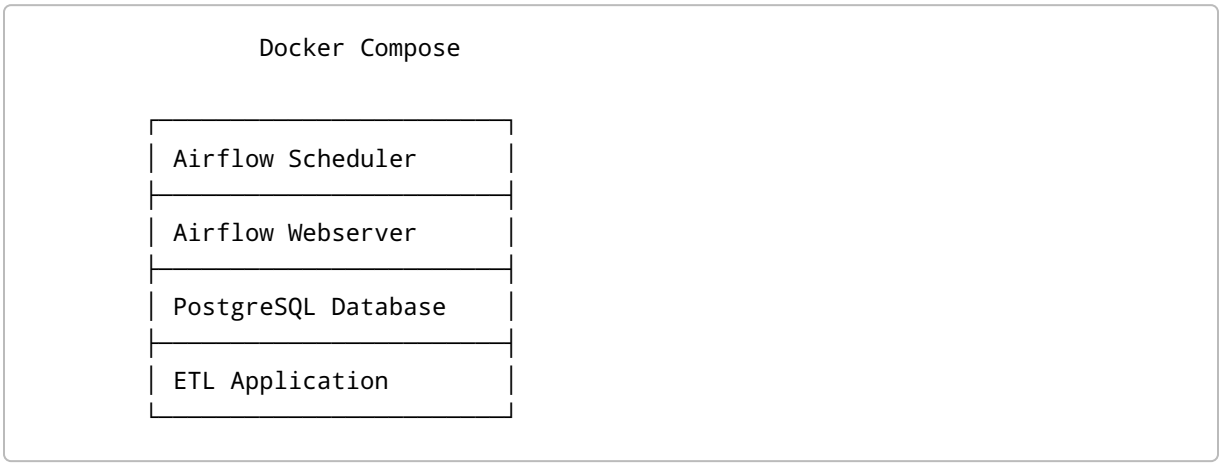
- Market Sentiment
- Top Movers
- Pipeline Health

Separating analytics tables from raw data improves performance and simplifies report design.

---

12. Deployment Architecture

The application is designed to run using Docker Compose.



Containers communicate through an internal Docker network.

Environment variables provide configuration and secrets.

---

13. Scalability

The architecture supports future expansion by:

- Adding new API connectors
- Adding additional processed tables
- Creating new analytics models
- Supporting more dashboards
- Scaling Airflow workers
- Partitioning large historical tables
- Introducing caching or streaming technologies

---

14. Security Considerations

The system follows basic security best practices.

- Store secrets in environment variables
- Use parameterized SQL queries
- Validate all external input
- Never expose database credentials
- Handle API failures safely
- Restrict dashboard access to analytics tables only

---

15. Design Decisions

Decision	Reason
Layered database architecture	Clear separation of raw, processed, and analytics data
PostgreSQL	Reliable relational storage with strong SQL support
Apache Airflow	Production-grade workflow orchestration
Incremental loading	Efficient historical data management
Python ETL modules	Flexibility and maintainability
Power BI	Rich business intelligence capabilities
Docker Compose	Consistent local development and deployment
Modular project structure	Easier testing, maintenance, and scalability

---

16. Future Architecture Enhancements

Potential improvements include:

- Kafka for real-time streaming
- dbt for transformation management
- Redis for caching
- Kubernetes for orchestration
- Prometheus and Grafana for monitoring
- CI/CD pipelines with GitHub Actions
- Cloud object storage for raw data archival
- Machine learning services for price prediction
- REST API for serving analytics

• Multi-environment deployments (Development, Staging, Production)